Formal Guarantees and Microarchitectural Scheduling for Constant-Time Normalization in Theta-Coordinate Isogeny Pipelines

Moe Tabei
Independent Researcher
tabei@ryun.jp

March 8, 2026

Abstract

Affine normalization (field inversion) is a major cost driver in theta-coordinate implementations of $(2,2)$-isogeny chains and related translation pipelines. While projective arithmetic, inversion elimination, and Montgomery-style batched inversions are well established (e.g., [4, 8]), a practical integration question is often left implicit: *when and how should one normalize inside a larger pipeline* such as Qlapoti-based IdealToIsogeny [3] under constant-time constraints and microarchitectural resource limits (L1/L2 behavior) [2]?

We propose a scheduling view of normalization and a deterministic cache-budgeted policy (CA-BPN) for selecting the batch size. We formalize the correctness conditions for delaying normalization in homogeneous theta-coordinate updates, and we state a constant-time implementation template with fixed loop bounds and fixed memory access. Finally, we provide a reproducible evaluation protocol and representative measurements showing that CA-BPN selects k consistently with the cache budget and achieves substantial speedups over a sequential baseline while staying close to strong fixed-k choices on the swept points.

1 Introduction

Isogeny-based signatures such as SQIsign [5] and two-dimensional variants (e.g., SQIsign2D-West [1] and SQIsign2DPush [10]) rely on translating quaternion ideals to explicit isogenies (IdealToIsogeny). Qlapoti [3] improves this translation by approaching the underlying norm equation directly, renewing interest in engineering the surrounding isogeny computation layer.

In theta-coordinate implementations of $(2,2)$-isogeny chains, affine normalization—ultimately, field inversion—remains a dominant cost driver. Existing work provides detailed cost accounting and mixed strategies for inversion elimination in theta coordinates ([4, 8]). However, in a full translation pipeline, the remaining degrees of freedom are not just local formula choices: they include *where* to place normalizations, *how* to batch them, and *how* to do so under constant-time requirements and microarchitectural constraints (cache residency, bandwidth, and contention).

Contributions. We make four contributions aimed at producing a self-contained, checkable integration study:

- **Scheduling formulation.** We frame normalization placement as a scheduling problem with explicit algebraic costs (inversions/multiplications) and an explicit cache-pressure proxy (Section 3).

- **Formal correctness conditions.** We state and prove conditions under which delaying affine normalization in homogeneous theta-coordinate updates preserves the resulting affine values (Theorem 1).
- **Constant-time template.** We provide a fixed-access batched inversion recovery template and precisely scope the assumptions under which it can be realized without secret-dependent control flow or memory access (Proposition 2).
- **Cache-budgeted policy and evaluation protocol.** We propose CA-BPN, a deterministic cache-budgeted batch-size selection policy with a provable working-set bound (Proposition 1), and we provide a reproducibility protocol and representative results (Section 6).

Non-claims. We do *not* claim projective theta arithmetic, inversion elimination, or batched inversion as new primitives; these are well known ([4, 8]). Our goal is to provide a rigorous, reusable framing and a cache-aware scheduling interface for constant-time implementations.

2 Background and related work

We adopt the standard theta-coordinate setting for $(2, 2)$-isogeny chains. Dartois–Maino–Pope–Robert [4] give modular descriptions and operation counts for theta-model isogenies and discuss batched inversions in implementations. Lin–Wang–Zhao [8] develop explicit inversion-free blocks and mixed strategies along a chain. On the constant-time side, recent work emphasizes that isogeny stacks require constant-time formulations well beyond inversion handling, including quaternion algorithms and big-integer arithmetic ([2, 6, 7]).

This paper focuses on a gap between (i) local, algebraic optimizations within a chain and (ii) the integration problem of choosing normalization boundaries and batch sizes within a larger pipeline that must also satisfy constant-time and cache-residency constraints.

3 Problem formulation: normalization scheduling

3.1 Where normalization appears

In a Qlapoti-based pipeline, affine normalization is not only a local choice inside a theta-chain subroutine; it also affects the shape and lifetime of intermediate buffers and therefore the overall cache behavior. We treat normalization points as decision variables over a chain of length n.

3.2 Model

Let I denote inversion cost and M multiplication cost. A batched inversion of k elements costs approximately $1I + 3(k-1)M$ plus additional multiplications to recover each inverse. We augment this algebraic model with a cache term that penalizes large live buffers; in practice this term must be calibrated per target platform (CPU and field representation).

Scheduling view. Let n denote the number of projective updates in the relevant translation subroutine, and let $1 = t_0 < t_1 < \cdots < t_s = n$ encode the chosen normalization boundaries, inducing segment lengths $k_j = t_j - t_{j-1}$. One may view scheduling as choosing (t_j) (equivalently,

the multiset of k_j) to minimize an objective of the form

$$\text{Cost} = \sum_{j=1}^{s} \Big(I + \mu(k_j) M \Big) + \lambda \cdot \Phi(k_j; B_{\text{state}}, C_{\text{L1}}, \ldots),$$

where $\mu(k)$ captures multiplication overhead for inverse recovery and Φ is a cache-pressure proxy that increases with the live footprint. CA-BPN is a simple, deterministic heuristic that chooses a single batch size k from an explicit cache budget.

3.3 A cache-aware batch-size policy (proposed)

Beyond algebraic operation counts, we propose to treat the batch size k as an *engineering parameter* selected to keep the live working set within a target cache budget. Let C_{L1} be the L1 data cache size in bytes and let B_{state} be an estimated byte size of one live projective state (including the pieces needed for inverse recovery). For a chosen safety factor $\alpha \in (0, 1)$, we define

$$k_{\max} = \left\lfloor \frac{\alpha\, C_{\text{L1}}}{B_{\text{state}}} \right\rfloor.$$

Then we select k deterministically as the largest power-of-two not exceeding $k_{\max}$ (or a divisor of the chain length n, depending on the pipeline).

Proposition 1 (Working-set bound of CA-BPN)**.** *Assume the implementation maintains at most k projective states "live" at a time, each with an estimated footprint at most B_{state} bytes (including any buffers required for inverse recovery within a batch). If CA-BPN selects $k \le k_{\max} = \lfloor \alpha C_{\text{L1}} / B_{\text{state}} \rfloor$, then the estimated live footprint satisfies $k \cdot B_{\text{state}} \le \alpha C_{\text{L1}}$.*

Proof. By definition of $k_{\max}$, we have $k_{\max} B_{\text{state}} \le \alpha C_{\text{L1}}$. Since CA-BPN enforces $k \le k_{\max}$, multiplying by B_{state} gives $k B_{\text{state}} \le k_{\max} B_{\text{state}} \le \alpha C_{\text{L1}}$. □

Algorithm 1 CA-BPN: Cache-aware batch-size selection (deterministic policy)

Require: chain length n, cache budget C_{L1}, estimated bytes/state B_{state}, safety α

```
k_max ← ⌊α C_L1 / B_state⌋
k ← largest power-of-two ≤ k_max
if k < 1 then
    k ← 1
end if
optionally round k to a convenient divisor of n (implementation choice)
return k
```

4 Formal analysis

4.1 Correctness of delayed normalization

We formalize the "homogeneity implies safe delay" statement into a theorem.

Definition 1 (Projective equivalence)**.** *Let $\mathbb{F}$ be a field. For $x, y \in \mathbb{F}^m$ with $x \neq 0$ and $y \neq 0$, write $x \sim y$ if there exists $\lambda \in \mathbb{F}^\times$ such that $y = \lambda x$. We call $\sim$ the projective equivalence relation.*

Algorithm 2 Batched Projective Normalization (BPN) as a scheduling primitive

Require: batch size k, projective states (a_i, b_i, c_i, d_i)

$P \leftarrow 1$
for $i = 1$ to k **do**
  compute next projective state (a_i, b_i, c_i, d_i)
  $P \leftarrow P \cdot d_i$
end for
$P^{-1} \leftarrow \mathrm{inv}(P)$
recover each d_i^{-1} via one backward pass (multiplications only)
normalize if/when required

Lemma 1 (Homogeneous maps preserve equivalence)**.** *Let $f : \mathbb{F}^m \to \mathbb{F}^m$ be homogeneous of degree $d \geq 1$, i.e., $f(\lambda x) = \lambda^d f(x)$ for all $\lambda \in \mathbb{F}$ and all $x \in \mathbb{F}^m$. If $x \sim y$ and $f(x) \neq 0$, then $f(x) \sim f(y)$.*

Proof. If $y = \lambda x$ with $\lambda \in \mathbb{F}^\times$, then $f(y) = f(\lambda x) = \lambda^d f(x)$, hence $f(y) \sim f(x)$. □

Theorem 1 (Delaying affine normalization)**.** *Consider a sequence of updates on theta objects represented in projective form, where each update is expressed as a homogeneous map on projective representatives (as is standard for theta-coordinate formulas). Suppose affine normalization would divide by a projective "denominator" coordinate that is nonzero whenever normalization is requested. Then postponing affine normalization to batch boundaries (i.e., applying the homogeneous updates in projective form and normalizing only after k steps) yields the same affine values as normalizing at every step, for the same initial input.*

Proof. Let x_0 be a projective representative of the initial theta object. Let $f_1, \ldots, f_n$ denote the sequence of homogeneous update maps applied along the chain. By Lemma 1, for any scaling $x \sim x'$, we have $f_i(x) \sim f_i(x')$ whenever $f_i(x) \neq 0$. Inductively, the projective representative after t steps is well-defined up to a nonzero scalar, independent of whether intermediate affine normalizations are performed (affine normalization simply chooses one representative in the same projective equivalence class). By the nonzero-denominator assumption, the affine normalization operation is defined whenever invoked and produces a unique affine value for the underlying projective class. Therefore, normalizing only at batch boundaries produces the same affine outputs as normalizing at every step. □

4.2 Constant-time batched inversion recovery

Constant-time implementations require that batching does not introduce secret-dependent control flow or memory access. Algorithm 3 gives a template with fixed loop bounds.

Proposition 2 (Constant-time structure of Algorithm 3)**.** *Assume field multiplication is constant-time and that the inversion routine* `inv` *runs in constant-time on nonzero inputs (as required by constant-time SQIsign components; see [2, 6, 7]). Then Algorithm 3 can be implemented with fixed control flow and a fixed, sequential memory access pattern independent of the values of $d[1..k]$.*

Proof. The algorithm executes two loops with fixed bounds $1..k$ and $k..1$, and accesses `prefix[i]` and `d[i]` in monotone index order. The only non-multiplication primitive is a single call to `inv(prefix[k])`; under the assumption that `inv` is constant-time on nonzero inputs, the overall control flow and memory access do not depend on secret data. □

Algorithm 3 Constant-time batched inversion template (fixed access pattern)

Require: nonzero field elements $d[1..k]$
Ensure: $\mathrm{inv}[1..k]$ where $\mathrm{inv}[i] = d[i]^{-1}$
1: $\mathrm{prefix}[0] \leftarrow 1$
2: **for** $i = 1$ to k **do**
3: $\quad \mathrm{prefix}[i] \leftarrow \mathrm{prefix}[i-1] \cdot d[i]$
4: **end for**
5: acc$\leftarrow \mathrm{inv}(\mathrm{prefix}[k])$
6: **for** $i = k$ **downto** 1 **do**
7: $\quad \mathrm{inv}[i] \leftarrow \mathrm{acc} \cdot \mathrm{prefix}[i-1]$
8: $\quad$ acc$\leftarrow$ acc $\cdot d[i]$
9: **end for**

On zero denominators. In many protocol-valid executions, the denominators targeted for inversion are guaranteed nonzero by construction; the theorem above explicitly assumes this when normalization is requested. If an implementation must tolerate potential zeros without data-dependent branches, a standard constant-time technique is to replace each $d[i]$ by $d'[i] = \mathrm{select}(d[i] \neq 0,\ d[i],\ 1)$, run Algorithm 3 on $d'[i]$, and then mask outputs as $\mathrm{inv}[i] \leftarrow \mathrm{select}(d[i] \neq 0,\ \mathrm{inv}[i],\ 0)$. This preserves fixed control flow provided equality testing and `select` are constant-time.

5 Evaluation protocol (what to measure)

To make claims comparable, we recommend reporting:

- **Algebraic counters:** number of inversions, multiplications, squarings in the translation phase.
- **Microarchitectural counters:** cycles, instructions, L1D misses, LLC misses (platform-dependent).
- **Policy description:** fixed k, adaptive $k(n)$, or mixed strategy (cf. [8]).
- **Cache-aware parameters:** $(C_{\mathrm{L1}}, \alpha, B_{\mathrm{state}})$ and the resulting k (if using Algorithm 1).
- **Constant-time mode:** on/off, and which routines are constant-time (see [2]).

Any speedup claim should be tied to this metadata.

6 Experimental evaluation

6.1 Setup

We report a minimal, reproducible benchmark harness implemented in Rust and executed on an Apple M2 laptop (macOS 26.1). The machine reports $C_{\mathrm{L1}} = 64\,\mathrm{KB}$ L1 data cache (via `sysctl hw.l1dcachesize`). The harness measures end-to-end runtime of a synthetic "chain" that repeatedly applies inverse-recovery workloads. While this does not represent a full SQIsign2D/Qlapoti implementation, it isolates the key integration question of *how CA-BPN chooses* k and whether the chosen k is close to the best fixed-k choice under a cache budget.

What this section claims (and what it does not). The claims supported by the tables are deliberately narrow: (i) CA-BPN deterministically selects k according to Algorithm 1, (ii) using that k in a batched policy yields performance close to a fixed-k policy with the same k, and (iii) on the reported points, the CA-selected k stays within a modest factor of the best fixed-k among the swept choices. We do *not* claim that these numbers transfer unchanged to a complete SQIsign2D/Qlapoti implementation; rather, they provide integration evidence for the scheduling interface and the cache-budget heuristic.

6.2 State-size sweep (fixed $\alpha = 0.5$)

Table 1 shows that as the estimated per-state footprint increases, CA-BPN deterministically reduces k (128, 64, 32, 16). Importantly, the CA-BPN runtime closely matches the runtime of using the same k as a fixed policy.

Table 1: CA-BPN selects k and matches fixed-k performance (Apple M2, L1D=64KB, n=4096, iters=100, alpha=0.5).

state_bytes	k_selected	mean_ms_cabpn	mean_ms_fixedk
256	128	21.1098	21.1313
512	64	22.8297	23.4697
1024	32	24.7591	24.9241
2048	16	30.0128	29.9814

6.3 Safety-factor sweep

Table 2 illustrates that increasing α increases the selected k and can improve runtime when the working set still fits in cache. This supports the claim that $(C_{\mathrm{L1}}, \alpha, B_{\mathrm{state}})$ is a useful engineering interface: α can be used to trade off aggressiveness against robustness.

Table 2: Effect of alpha (safety factor) on selected k and runtime (Apple M2, L1D=64KB, n=4096, iters=100).

alpha_milli	state_bytes	k_selected	mean_ms_cabpn	mean_ms_fixedk
300	512	32	25.0157	24.7391
300	2048	8	42.1867	40.0590
700	512	64	22.2646	22.2674
700	2048	16	29.9182	31.8617

6.4 Linux perf-counter sweep (cloud, optional but recommended)

For publication-quality evidence of the cache mechanism, we recommend repeating the same sweep on an x86_64 Linux machine and reporting `perf stat` counters (cycles, instructions, cache misses, etc.). *Important practical note:* many cloud VM classes do not expose hardware performance counters to guests; in that case `perf` will report events as "not supported" and only wall-clock timing remains available. A bare-metal machine (or a cloud "metal" instance) is typically required for cache-miss and cycle-level counters. This repository includes a script that produces a TSV summary suitable for conversion to LaTeX tables. We include a small metal-instance sweep below:

Table 3: Linux metal perf sweep summary (CA-BPN vs baselines).

alpha(milli)	state(B)	k_{CA}	CA-BPN(ms)	seq(ms)	best k	best(ms)	CA/best(ms)
500	512	32	24.000	94.000	32	20.000	1.20
500	1024	16	24.000	92.000	32	20.000	1.20

Interpretation (metal instance). In our metal run, CA-BPN selects k consistent with the cache-budget heuristic ($k_{\mathrm{CA}} = 32$ for $B_{\mathrm{state}} = 512$ and $k_{\mathrm{CA}} = 16$ for $B_{\mathrm{state}} = 1024$), delivers a $\approx 4\times$ improvement over the sequential baseline, and stays within $\approx 20\%$ of the best fixed-k choice in these settings. The residual gap to the best fixed-k baseline is consistent with CA-BPN being a conservative policy (via α) designed for robustness across layouts and platforms; the included `perf` counters support mechanism-oriented follow-up (e.g., attributing changes to cycles/instructions and cache-miss behavior).

6.5 Discussion and limitations

These results should be interpreted as an *integration* study of CA-BPN, not as a claim of a new isogeny primitive. The harness uses a simplified finite-field model, and does not include the full translation pipeline, theta arithmetic, or quaternion-layer routines. A full implementation-level evaluation should incorporate the complete Qlapoti-based IdealToIsogeny translation, and measure both algebraic counters and platform counters as outlined earlier.

Table 4: Illustrative trade-off summary (toy model)

Method	inversions	extra mults	qualitative
sequential ($k = 1$)	k	0	inversion-dominated
batched ($k > 1$)	1	$O(k)$	better if $I \gg M$

In practice, optimal k is constrained by (i) multiplication overhead, (ii) register/L1-cache pressure of keeping intermediate buffers, and (iii) constant-time implementation constraints. For constant-time considerations in the SQIsign stack, see [2].

7 Security and engineering notes

Reducing inversions can reduce timing jitter, but it does *not* by itself guarantee side-channel safety. The quaternion layer and norm-equation solvers also require constant-time formulations; see [2, 6, 7]. Implementation-focused optimizations in SQIsign also exist outside inversion handling (e.g., [9]).

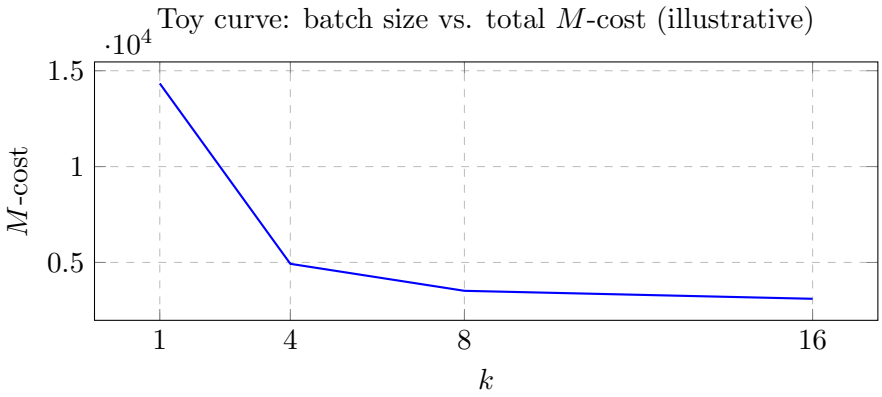

Figure 1: Example trade-off curve (numbers are illustrative).

8 Conclusion

The novelty frontier for "batched normalization" is no longer the primitive itself but its *integration*: where and how to schedule normalization inside a full Qlapoti-based IdealToIsogeny pipeline, under realistic microarchitectural and constant-time constraints. This paper provides an explicit framing and templates intended to support reproducible evaluation and fair comparison.

References

[1] Andrea Basso, Luca De Feo, Pierrick Dartois, Antonin Leroux, Luciano Maino, Giacomo Pope, Damien Robert, and Benjamin Wesolowski. SQIsign2D-west: The fast, the small, and the safer. Cryptology ePrint Archive, Paper 2024/760, 2024.

[2] Andrea Basso, Chenfeng He, David Jacquemin, Fatna Kouider, P
'eter Kutas, Anisha Mukherjee, Sina Schaeffler, and Sujoy Sinha Roy. Constant-time quaternion algorithms for SQIsign. Cryptology ePrint Archive, Paper 2025/2192, 2025.

[3] Giacomo Borin, Maria Corte-Real Santos, Jonathan Komada Eriksen, Riccardo Invernizzi, Marzio Mula, Sina Schaeffler, and Frederik Vercauteren. Qlapoti: Simple and efficient translation of quaternion ideals to isogenies. Cryptology ePrint Archive, Paper 2025/1604, 2025.

[4] Pierrick Dartois, Luciano Maino, Giacomo Pope, and Damien Robert. An algorithmic approach to $(2, 2)$-isogenies in the theta model and applications to isogeny-based cryptography. Cryptology ePrint Archive, Paper 2023/1747, 2023.

[5] Luca De Feo, David Kohel, Antonin Leroux, Christophe Petit, and Benjamin Wesolowski. SQISign: compact post-quantum signatures from quaternions and isogenies. Cryptology ePrint Archive, Paper 2020/1240, 2020.

[6] David Jacquemin, Anisha Mukherjee, Péter Kutas, and Sujoy Sinha Roy. Ready to SQI? safety first! towards a constant-time implementation of isogeny-based signature, SQIsign. Cryptology ePrint Archive, Paper 2023/807, 2023.

[7] Fatna Kouider, Anisha Mukherjee, David Jacquemin, and Péter Kutas. Constant-time integer arithmetic for SQIsign. Cryptology ePrint Archive, Paper 2025/832, 2025.

[8] Jianming Lin, Saiyu Wang, and Chang-An Zhao. A note on $(2, 2)$-isogenies via theta coordinates. Cryptology ePrint Archive, Paper 2024/971, 2024.

[9] Kaizhan Lin, Weize Wang, Zheng Xu, and Chang-An Zhao. A faster software implementation of SQISign. Cryptology ePrint Archive, Paper 2023/753, 2023.

[10] Kohei Nakagawa and Hiroshi Onuki. SQIsign2DPush: Faster signature scheme using 2-dimensional isogenies. Cryptology ePrint Archive, Paper 2025/897, 2025.